RL for Wordle

CS-5410: Reinforcement Learning

Project Report

Saptarishi Dhanuka, Divij Khaitan

April 12, 2026

Contents

1 Introduction

The game we chose to solve was the popular Wordle. The goal of the game is to guess a mystery five-letter word in six guesses, based on limited feedback. The rules of the feedback are:

1. Any letter in the guess placed in the same position as it would be in the target is marked in green
2. Any letter present in the target that is incorrectly placed in the guess is marked in yellow
3. In the case of duplicate misplaced letters, the first n occurences of the letter in the guess will be marked yellow if the letter appears n times in the target
4. Any letters absent from the word or excess duplicate letters are marked in black/grey

There is a list of about 2000 words that the game chooses a target from daily. The goal of the project is to train an agent to win consistently and quickly at this game, using reinforcement learning techniques.
As we talk about in following sections, we first tried training agents from scratch to predict the best possible word, but those did not give good results at all due to the large search space of possible words. Hence we switched to a meta-learning approach with "heuristic agents", which instead learnt to choose from a set of deterministic heuristics, which gave us much better performance.

While we used different environments while formalizing the problem due to our different approaches, it can be broadly described as an MDP with the following components.

- **State Space**: Every possible state of the board i.e. the combination of the guesses made alongside the feedback, modulo permutations of letters and guess orders. To ensure a consistent ordering, the guesses may be sorted in alphabetical order. Moreover, the Markovian property comes from the fact that each state encodes all the words that have been guessed so far and the feedback that was received from each guess.
  This state space was used for training agents/algorithms from scratch for trying to predict the next word, which proved to be infeasible. The state space for the meta-learning approach was the feedback received at each step in terms of the number of greens, yellows and greys.
- **Action Space:** For the from-scratch agents, the actions were the next word to predict, while for the meta-learners it was which heuristic to pick.
- **Rewards:** The most basic reward structure is to reward the agent +10 for winning the game and -10 for losing. We experimented with a variety of reward structures that inlcuded intermediate rewards and penalities for greens, yellows and greys, which we explain in section 4.
- **Transition Probabilities:** The transitions for this game are completely deterministic, moving from a board state with k guesses made to one with $k+1$ guesses made, with the feedback being decided by the rules outlined above.
- **Gamma:** A goal of the game is speed alongside correctness, gamma should be set to less than 1.

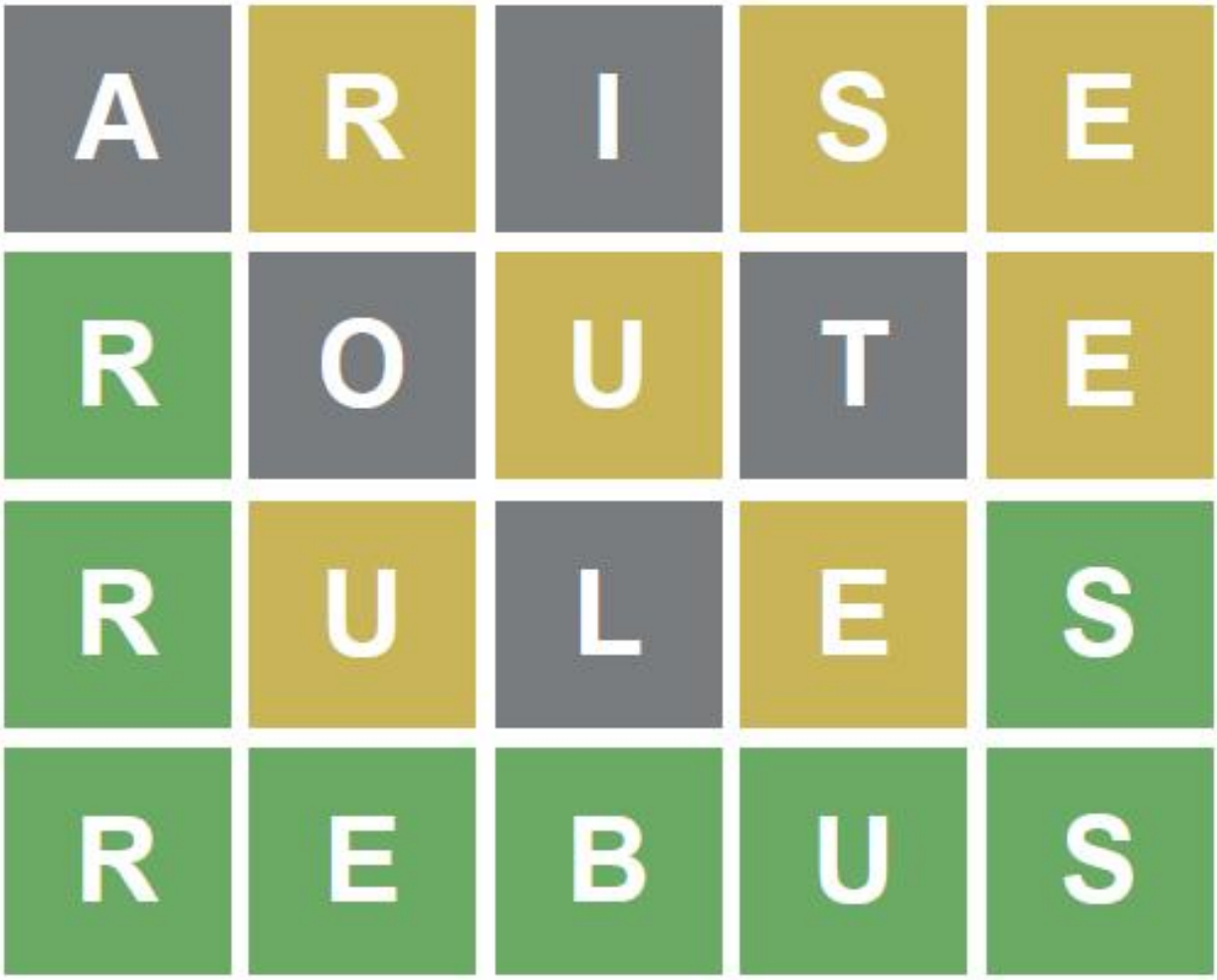

Figure 1: Example game of wordle

2 Literature Review

Bhambri, Bhattacharjee and Bertsekas [4] use RL to attempt to improve upon a base heuristic, eliminating all impossible words from consideration as the guesses are rolled out. This is done by computing the word which has the least average distance from the entire remaining list under the heuristic. Sanderson [1] uses information theory to maximise the gain of information on each subsequent guess based on the entropy of words in the list of target words. Anderson [2] trains an agent to select the best heuristic for every situation from a set of heuristics. Short [7] guesses the word with the maximum posterior probability based on all known information upto that point.

3 Data

The word list for wordle can be found on [3]. The game allows around 14000 valid words to be guessed, however only 2309 of the words will actually be chosen to be the target. Our experiments primarily focus on the set of 2309 words due to computational constraints, while a few of our experiments inlcude the full set of 14000 words.

We didn't carry out any extensive data exploration and analysis since our base dataset was a simple enough list of words. We also didn't want to extract statistics from the dataset and use an information theoretic approach, since that would determinisitcally brute force the probem without using any reinforcement learning.

4 Methods & Experimentation

This will be broken down into 3 sections: Agents, Algorithms, and Reward Structures

4.1 Agents

4.1.1 Simple Agent

This agent uses the board as-is to encode the state of the process. This is used as input to a Q-table or neural network to obtain the estimated Q value or the next action.
This is in the file ppo_agents.py, under the name WordleFeatureExtractor.

4.1.2 Information Encoding Agent

This agent encodes the feedback in a permutation invariant manner. The state of the game is represented using a 5×26 matrix, with each row representing the known information about a single letter and each column representing the known information about a single position. The rules for the encoding are as follows

1. The matrix is initialised with all 0s. This represents a lack of information
2. Any green feedback has the entry for the letter at that position in the matrix changed to a +1, with that same entry for every other letter being marked with -1.
3. For yellow feedback, in addition to knowing a lower bound on the number of occurences of the letter, we can also eliminate any position that has already been marked green, already been marked yellow with that letter or already been marked black with that letter. A -1 is be placed at every position we can eliminate for that letter, and the candidate positions are filled with the value

$$\frac{\#(\text{Number of Known Occurences of Letter})}{\#(\text{Number of Candidate Positions})}$$

   This has the upshot of giving us the correct positions with enough elimination.
4. For black feedback, if the letter has no other feedback in the guess, it is marked as -1 indicating absence across all 5 positions. If not, only the position in the guess is marked as -1.

This is in ppo_agents.py, under the name WordleFeatureExtractorMarkov.

4.1.3 Meta-Learner Agent

We were facing difficulties with trying to make agents and algorithms learn from scratch due to the large search space of the problem and the variety of different strategies that agents could possibly learn, so we decide to use a meta-learning strategy.

Here the agent has access to a number of different deterministic heuristics and it has to learn to pick the best heuristic instead of learning from scratch what word to predict next. Loosely, heuristics can be defined as simplified rules of thumb that may not always be applicable and no formal guarantees can be made about. Examples from chess would be rules taught to beginners such as not moving the same piece multiple times in the opening, controlling the opponent's access to the central squares and not pushing pawns in front of a castled king.

The agent's actions were to choose a particular heuristic. After reducing the search space through the heuristics, we then just picked words randomly from this smaller search space, which gave surprisingly good performance as shown in the results. The heuristics include

1. **Random**: Choosing a word at random
2. **Without absence**: Ensuring that any letter-position pairs known to be impossible (black feedback) are missing from the next action by filtering through words appropriately
3. **Confirmed without absence**: The same as the above, with the additional stipulation that any confirmed letters (green feedback) must be at the correct position in the next action
4. **Misplaced and Confirmed without absence**: Ensuring that missing letters are absent, confirmed letters are present and misplaced letters (yellow feedback) are incorporated in accordance with their counts. With regards to the misplaced/yellow letters, it kept only words that contain every yellow letter somewhere.
5. **Letter frequency:** By finding empirical distribution of letters in the English language, we score each word in the target word list by counting how many common English letters it contains (plus a small bonus for overall uniqueness).
6. **Green, Yellow and Grey Information Incorporated**: This was the most involved heuristic where it was mostly similar to heuristic 4, but with the added constraint that we ban yellow letters from the specific indices where they were previously guessed.

This is in the file myQWordleEnv.py.

4.2 Algorithms

4.2.1 Q-Learning

The goal of this is to estimate a table $Q(s, a)$, where the Q-function is the expected reward of taking action a while in state s. Since wordle is a finite horizon problem of at most six steps, the table has an additional index h for the number of steps. The table is updated using the Bellman equations.

$$Q_h(s, a) := Q_h(s, a) + \alpha(\mathbb{E}_{s'}[r + \gamma(\max_{a'} Q_{h+1}(s', a') - Q_h(s, a)|(s, a))]$$

The table is initialised with all of the entries having value 0. Ties between actions are broken arbitrarily, and an ϵ-greedy strategy is used to balance exploration with exploitation. [8]

For the from-scratch agent we didn't get any good performance due to the explosion in the state space as shown in Figure 2. The training reward also barely showed any improvement, as shown in Figure 3
For the heuristic agent, we use a form of Temporal Difference (TD) Learning where the exact update rule is given as below

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

where the states are the greens, yellows and greys and the actions are which heuristic to pick.

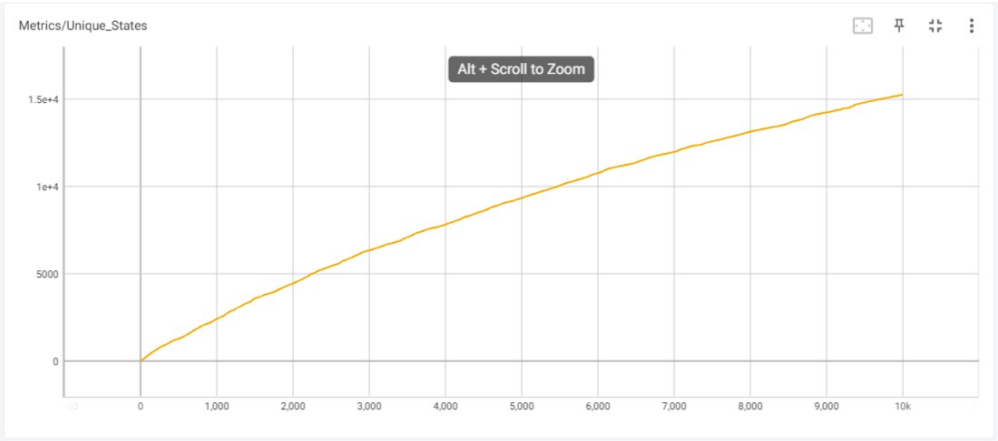

Figure 2: Explosion of Q states

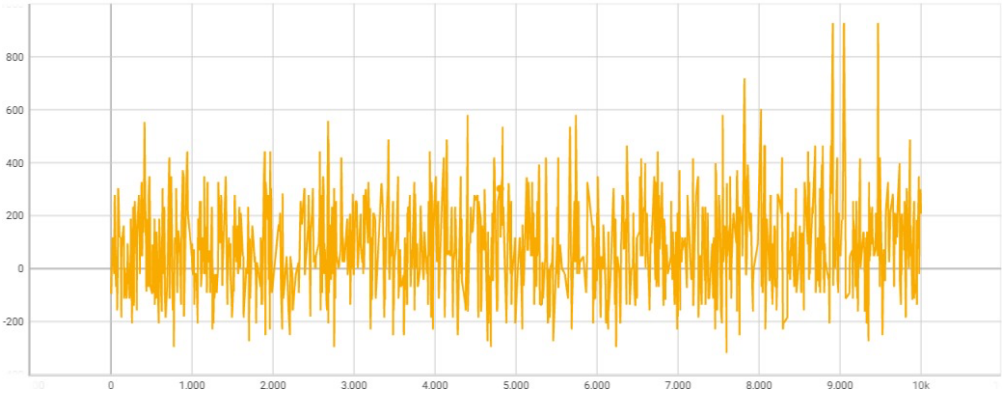

Figure 3: Training Reward for Q learning on the from-scratch Environment

4.2.2 Deep Q-Networks

For large state spaces, such a table quickly grows in size to the point where computing it explicitly becomes intractable. The solution to this is approximating the q table using a neural network, or $Q(s, a; \theta) \approx Q^*(s, a)$. However, this cannot be done in a supervised because the optimal q function is neither known nor can be sampled from. To do this, we need to be able to able to estimate the Q value, which becomes a chicken-and-egg problem. The solution to this is quite simple, to use the weights from the previous iteration to estimate the target q-values. Formally, the loss function and parameters in the i^{th} iteration are indexed as L_i and θ_i, then

$$L_i(\theta_i) = \mathbb{E}_{s'}[(y_i \ - \ Q(s, a; \theta_i))^2]$$

Where $y_i = \mathbb{E}[r + \gamma(max_{a'}Q(s', a'; \theta_{i-1}))|(s, a)]$. The parameters θ_{i-1} are frozen when optimising to find the best θ_i. Differentiating the gradient gives

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}[r + \gamma(\max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i))\nabla_{\theta_i} Q_i(s, a, \theta_i)]$$

When using SGD, updating the weights after every forward pass replaces the expectations with samples, and gives us back the Q-learning update. This approach is much better in high dimensional state spaces and allows for generalisation, in exchange for increased training variance. [5]

4.2.3 Proximal Policy Optimisation

This algorithm belongs to the class of algorithms called policy gradient algorithms. These are built around the notion of an advantage function, defined as

$$A(s, a) = Q(s, a) - V(s)$$

It captures how much better an action is in comparison to the alternatives. The common policy gradient method is to differentiate the objective $L = \hat{\mathbb{E}}[\log \pi_\theta(a_t|s_t)\hat{A}(t)]$. However, minimising this loss doesn't work very well in practice. Another important method is the trust region policy optimisation, which attempts to maximise the surrogate objective

$$\max \quad \hat{\mathbb{E}}_t[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\hat{A}_t] \qquad \text{(surrogate objective function)} \tag{1}$$

$$\text{subject to} \quad D_{KL}(\pi_{\theta_{old}}(.|s_t)||\pi_\theta(.|s_t)) \leq \delta \quad \text{(distance constraint)} \tag{2}$$

With linearisation, this can be efficiently solved. The theoretical justification for the method performs an unconstrained optimisation of $\max \hat{\mathbb{E}}_t[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\hat{A}_t] - \beta D_{KL}(\pi_{\theta_{old}}(.|s_t)||\pi_\theta(.|s_t))$, However, this is very sensitive to β which is intractable to tune.
PPO makes two changes. It proposes a clipped objective which is expressed as

$$L^{clip} = \hat{\mathbb{E}}_t[\max(r_t(\theta)\hat{A}_t, clip(r_t(\theta), 1 - \epsilon, 1 + epsilon)\hat{A}_t]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$. This emulates the KL constraint by removing the incentive to make drastic policy updates.
The second major component is that to ensure fixed episode lengths, the algorithm samples trajectories of a fixed length and computes the advantage as the difference between the value of the trajectory and the value of the initial state, i.e.

$$\hat{A}_t = \gamma^{T-t}V(s_T) + \gamma^{T-t-1}r_{T-1} + \gamma^{T-t-2}r_{T-2} + \cdots + r_t - V(s_t)$$

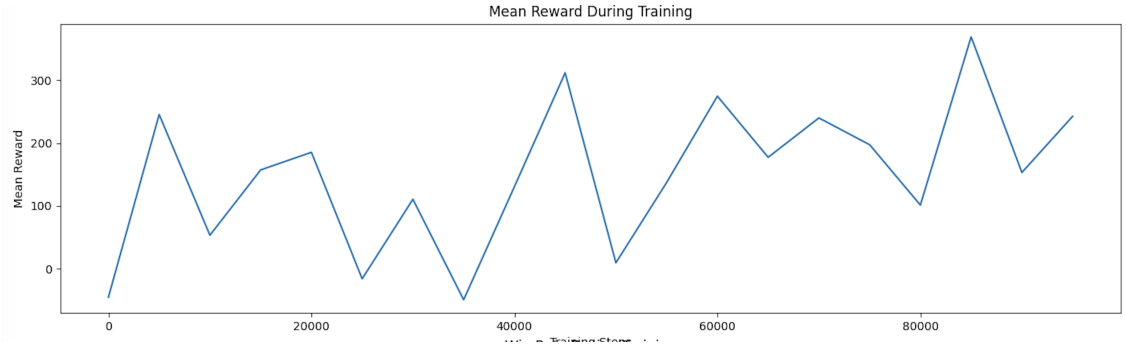

Figure 4: Training Reward for PPO

The agent is thus able to run several trajectories in parallel and sample different minibatches from them, which works very well for neural network agents.
For neural networks which usually share parameters for the policy and the value function, the surrogate loss is combined with the MSE of the the predicted value from the target value (similar to DQN) and the entropy of the output, to encourage exploration. Giving the loss [6]

$$L = \hat{\mathbb{E}}_t[L_t^{clip}(\theta) - c_1 L^{MSE}(\theta) + c_2 H(\pi_\theta(.|s_t))]$$

While experimenting with PPO in the from-scratch method, we were not able to get good results and even after training for even 100K iterations, the learnt model could not win a single game on evaluation. A graph of the mean training reward per episode is given in Figure 4, with rewards being as follows: win 100, green 100, yellow 10, grey -10. While we can see a small upward slope, it wasn't enough to win a single game in evaluation. More iterations may be needed for convergence.

4.3 Rewards

It is conventional to only reward the agent upon the completion of the game, however wordle lends itself to intermediate feedback and hence intermediate rewards. We tried assigning rewards based on the feedback, such as a +10 for a green, -1 for a black, + 2 for a yellow etc.. The other version we tried was the conventional method of rewarding the agent only upon completion, penalising losses and reinforcing wins.

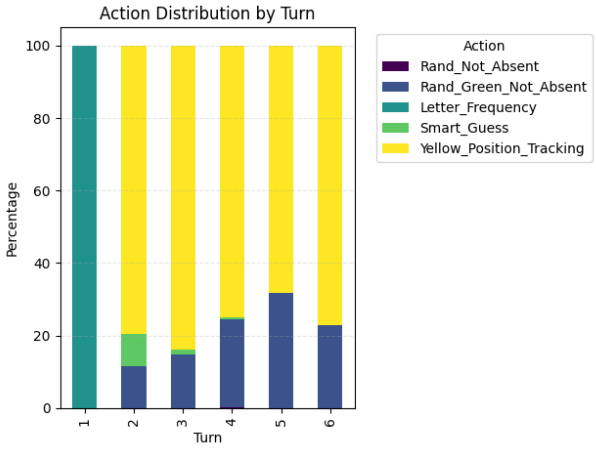

Figure 5: Action by Turn for Intermediate Rewards Setting

5 Results

Since the from-scratch agents were not able to perform well at all in our experiments, we have left them out of results. We focus on our successful Heuristic Learning approach.

5.1 Q-Learning Heuristic Method

5.1.1 Taking only target words

This method gave the best results where we got an average of **3.9 guesses** and 98 % success rate to finish the game when evaluated on the target words. We can see the distribution of guesses in Figure 6 and distribution of the actions chosen in Figure 5 which shows that the most complex heuristic called Yellow Position Tracking which takes into account the greens, yellows and greys is the best performing one and chosen most often. Note that 7 guesses means we lost the game after the permitted 6 guesses. Also to note is that the letter frequency heuristic is chosen in the first move when we don't have any feedback taken from the environment. It's interesting to note that the action named Smart Guess which takes also takes into account the greens, yellow and greys but in a less efficient manner is chosen less often than the action which takes into account just Greens and Greys. This may be because the Smart Guess just takes into account that yellow letters may appear somewhere in the word and not ban them from the positions where we know they aren't there, which doesn't really give us much information, while just taking into account greens and greys may be a simpler heuristic to eliminate more possible words.
Changing the reward formulation so that there we no intermediate rewards and only rewards for winning or cost for losing gave us similar performace as seen in Figure 7, but with a slightly worse average moves of **4.05**.

5.1.2 Taking all valid guess words

We also tried training the agent on all 14000 valid guess words and evaluate on the target words. This firstly took a lot more time than just training on targe words, and we got relatively worse results as well, as shown in Figure 8.

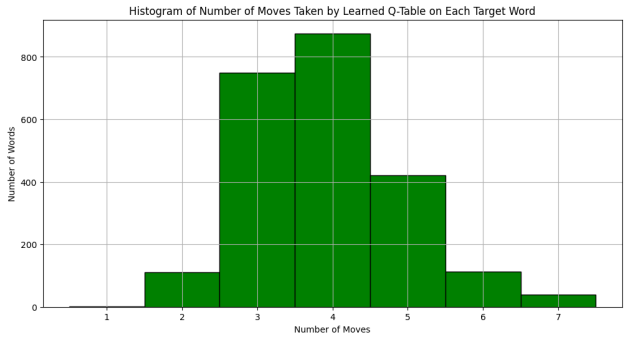

Figure 6: Q Learning with Heuristic Agent and Intermediate Rewards: Guess Distribution

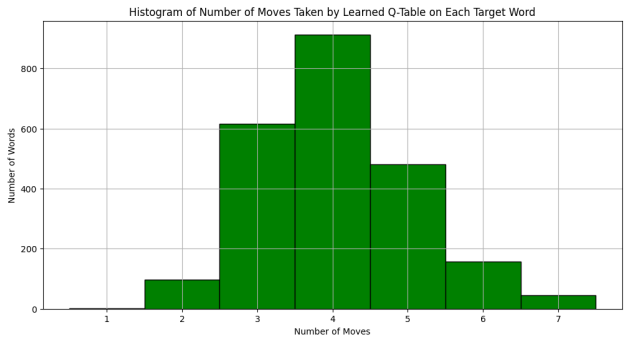

Figure 7: Q Learning with Heuristic Agent and No Intermediate Rewards: Guess Distribution

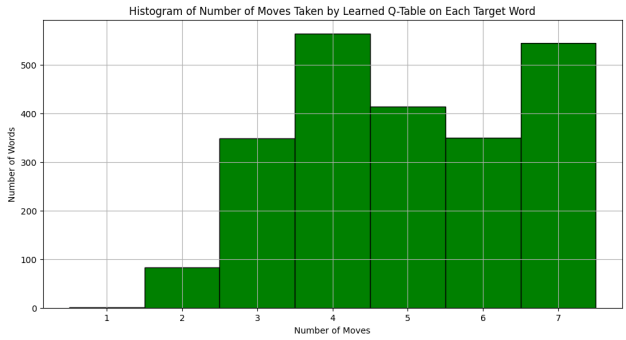

Figure 8: Q Learning with Heuristic Agent and Intermediate Reward when trained on all words

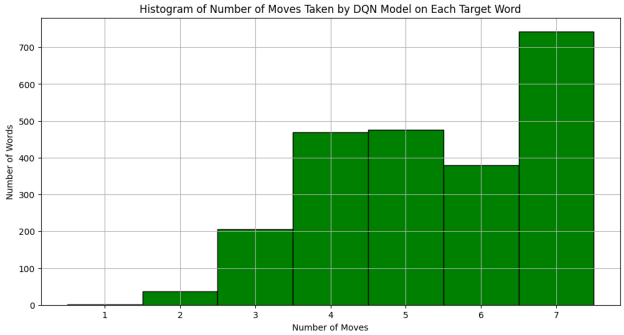

Figure 9: DQN with Heuristic Agents and Intermediate Rewards

5.2 DQN with Heuristic Agent

We also ran DQN from the stable baselines library on the heuristic agent, but it wasn't able to beat simple Q learning even with 100K iterations, as shown in Figure 9 and by an average guess length of 5.2. Even though this seems to be a more powerful method, the worse performance may simply be because we didn't run it for enough iterations.

6 Conclusion

We spent a lot of time formalising the problem for reinforcement learning and making the relevant environments. After trying to make agents and algorithms learn from scratch on our environments, we tried out a meta-learning approach with simple Q-learning and a new environment formalization, which gave us the best results. Our takeway from this would be that if the problem seems to be a relatively smaller one like in the Worlde case, where we just have 5 letter words and 6 guesses needed for each word, then it may be simpler to use simpler approaches like Q-learning to get good enough results. This would also save us the time that the more advanced methods like PPO and DQN from libraries would need for convergence.
Moreover, if a problem can be cut down to size by a few simple heuristics, then it may be a better choice to use a form of "meta-learning" where instead of learning an action in the environment from scratch, we instead learn to pick from a few man-made heuristics, which will abstract away the actual environment action and let us instead focus on picking from a much smaller set of actions.

It's of course important to acknowledge the limitations of our approach. One key issue is that we just use the 2309 target words given to us, which warrants retraining to any new set of words. Moreover, the simple Q-learning approach will not scale to larger search spaces as we have seen.

For our final project, we will still try to make an RL approach that tries to predict the next word from scratch, in addition to trying out more advanced methods for the meta-learning framework. We wil try to beat the best information theory based methods which take around 3.4 moves on average, and also extend the game to a larger environment where there could be longer allowed words and multiple words to be guessed.

References

[1] 3Blue1Brown. Solving wordle using information theory, 2022. Accessed: 2025-03-21.

[2] Benton J. Anderson and Jesse G. Meyer. Finding the optimal human strategy for wordle using maximum correct letter probabilities and reinforcement learning, 2022.

[3] Arjun Vikas (arjvik). Wordle wordlist, 2025. Accessed: 2025-03-21.

[4] Siddhant Bhambri, Amrita Bhattacharjee, and Dimitri Bertsekas. Reinforcement learning methods for wordle: A pomdp/adaptive control approach, 2022.

[5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning, 2013.

[6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

[7] Martin B. Short. Winning wordle wisely, 2022.

[8] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3):279–292, May 1992.